

Labolatorium 14

June 17, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Transformata Hough'a i Radona.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path
from skimage.feature import canny
from skimage.transform import hough_line, hough_line_peaks, radon, iradon, \
    hough_circle, hough_circle_peaks
from skimage.draw import circle_perimeter
from skimage.filters import gaussian
%matplotlib inline

def load_gray(path):
    """Wczytaj obraz w skali szarości jako numpy array."""
    return np.array(Image.open(path).convert('L'))

def norm(x):
    """Znormalizuj obraz do zakresu [0, 1]."""
    return (x - x.min()) / (x.max() - x.min() + np.finfo(float).eps)

def image_path(name):
    """Zwróć ścieżkę do pliku w katalogu lab14 (obsługa uruchamiania z repo-
    root lub z lab14)."""
    root = Path('lab14') if Path('lab14').exists() else Path('.')
    return root / name
```

2 Zadania

2.1 Zadanie 1.

Zamień obrazy `ez.bmp` i `dom.bmp` na obrazy konturowe, a następnie:

- Wykorzystaj funkcję `hough` do obliczenia transformaty Hough'a obrazu konturowego. Wyświetl transformatę w postaci 3D oraz jako obraz w poziomach szarości. Przyjmij kwant $ho = 2$ oraz kwant kąta $\theta = 1^\circ$. Następnie wyzeruj elementy transformaty poniżej 40% maksimum i ponownie wyświetl transformatę.
- Zapoznaj się z funkcjami `houghpeaks` i `houghlines`. Wykorzystaj je do aproksymacji konturów odcinkami prostych. Przetestuj różne liczby maksimumów.

2.1.1 a) Transformata Hougha

```
[2]: def edges_canny(img, sigma=1.5):  
    """Wykryj krawędzie operatorem Canny'ego."""  
    return canny(img / 255.0 if img.max() > 1 else img, sigma=sigma)  
  
def hough_transform(edge_img, theta_step=1, rho_step=2):  
    """Oblicz transformatę Hougha z kwantem kąta theta_step (°) i kwantem rho_  
↪rho_step (px)."""  
    angles = np.deg2rad(np.arange(-90, 91, theta_step))  
    rows, cols = edge_img.shape  
    diag = int(np.ceil(np.sqrt(rows**2 + cols**2)))  
    dists = np.arange(-diag, diag + rho_step, rho_step)  
    H = np.zeros((len(dists), len(angles)), dtype=np.float64)  
  
    y_idx, x_idx = np.nonzero(edge_img)  
    cos_a = np.cos(angles)  
    sin_a = np.sin(angles)  
    rho_vals = x_idx[:, None] * cos_a[None, :] + y_idx[:, None] * sin_a[None, :]  
    rho_indices = np.round((rho_vals - dists[0]) / rho_step).astype(int)  
  
    for i in range(len(angles)):  
        np.add.at(H[:, i], rho_indices[:, i], 1)  
  
    return H, angles, dists  
  
def plot_hough(H, angles, dists, title_prefix):  
    """Wyświetl transformatę Hougha jako obraz i wykres 3D."""  
    theta_deg = np.degrees(angles)  
    fig = plt.figure(figsize=(14, 5))  
    ax1 = fig.add_subplot(1, 2, 1)  
    ax1.imshow(H, aspect='auto', cmap='gray',  
               extent=[theta_deg[0], theta_deg[-1], dists[-1], dists[0]])
```

```

ax1.set_title(f'{title_prefix} - transformata Hougha')
ax1.set_xlabel('\u03b8 [°]')
ax1.set_ylabel('\u03c1 [piksel]')
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
T, D = np.meshgrid(theta_deg, dists)
ax2.plot_surface(T, D, H, cmap='viridis', edgecolor='none')
ax2.set_title(f'{title_prefix} - transformata Hougha (3D)')
ax2.set_xlabel('\u03b8 [°]')
ax2.set_ylabel('\u03c1 [piksel]')
ax2.set_zlabel('Akumulator')
plt.tight_layout()
plt.show()

```

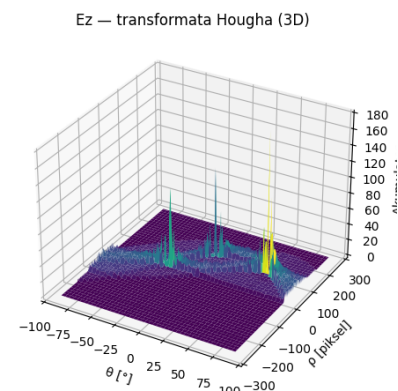
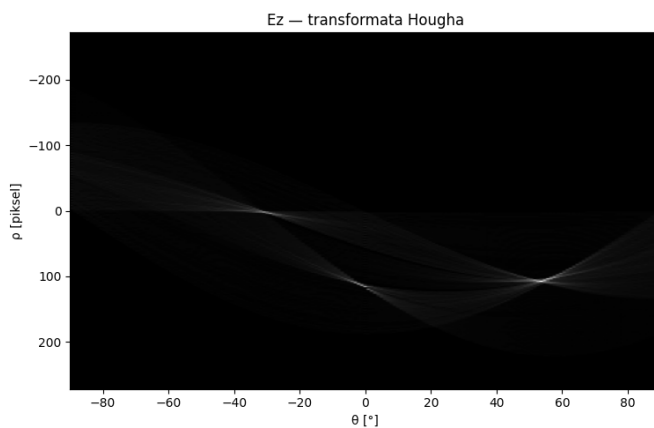
```

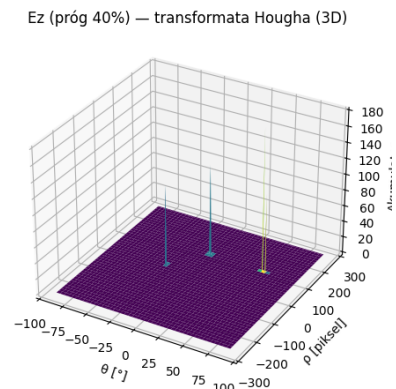
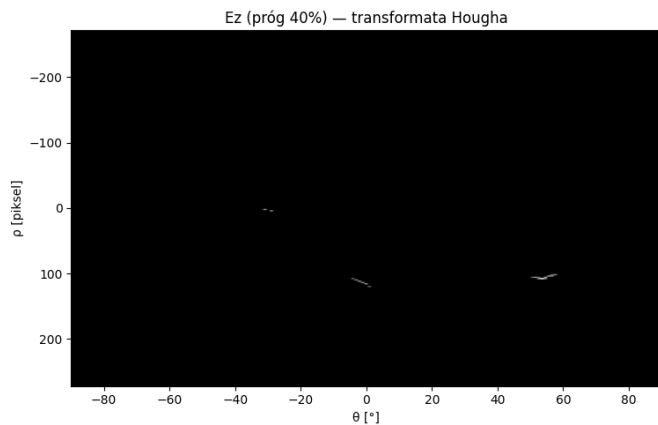
names = [('ez', 'ez.bmp'), ('dom', 'dom.bmp')]
for label, fname in names:
    img = load_gray(image_path(fname))
    edges = edges_canny(img)
    H, angles, dists = hough_transform(edges)
    print(f'{label}: kształt akumulatora = {H.shape}, max = {H.max()}')
    plot_hough(H, angles, dists, label.capitalize())

    H_thresh = H.copy()
    H_thresh[H_thresh < 0.4 * H.max()] = 0
    plot_hough(H_thresh, angles, dists, f'{label.capitalize()} (próg 40%)')

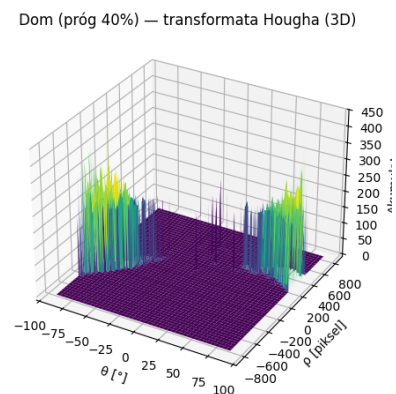
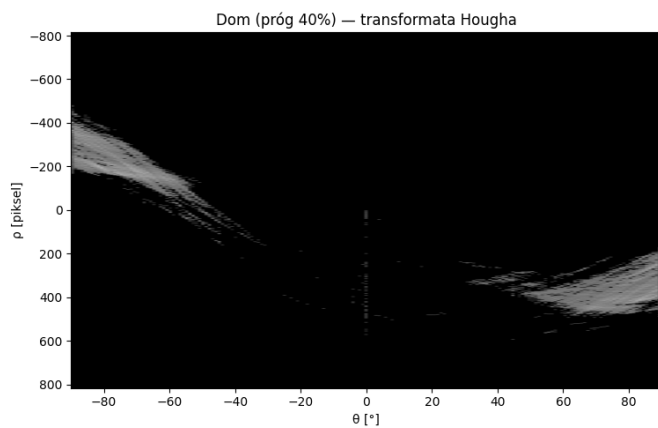
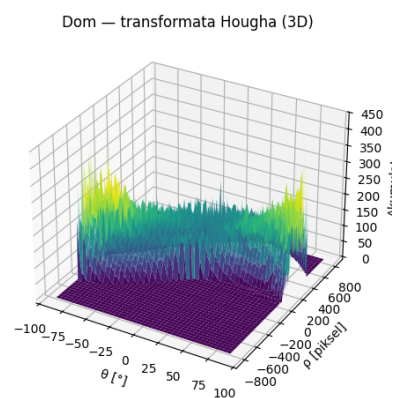
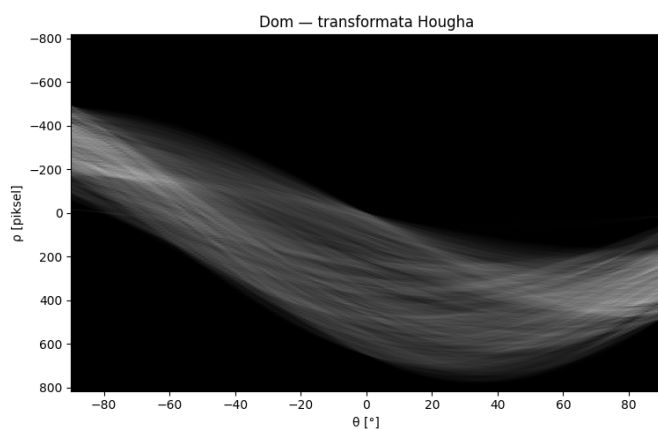
```

ez: kształt akumulatora = (273, 181), max = 179.0





dom: kształt akumulatora = (818, 181), max = 441.0



2.1.2 Wnioski

Najjaśniejsze punkty akumulatora odpowiadają prostym, które zawierają najwięcej pikseli krawędzi. Po progowaniu pozostają tylko dominujące kierunki, co ułatwia identyfikację głównych linii obiektu.

2.1.3 b) Detekcja prostych: houghpeaks i houghlines

```
[3]: def draw_hough_lines(ax, angles_peaks, dists_peaks, img_shape, color='red'):
    """Narysuj proste o postaci  $\rho = x \cos\theta + y \sin\theta$  na istniejącym
    ↪ axes."""
    h, w = img_shape
    for angle, dist in zip(angles_peaks, dists_peaks):
        cos_a, sin_a = np.cos(angle), np.sin(angle)
        if np.abs(sin_a) > 1e-3:
            x0, x1 = 0, w - 1
            y0 = (dist - x0 * cos_a) / sin_a
            y1 = (dist - x1 * cos_a) / sin_a
        else:
            y0, y1 = 0, h - 1
            x0 = x1 = dist / cos_a
        ax.plot([x0, x1], [y0, y1], color=color, linewidth=1.5)

def compare_peaks(edge_img, H, angles, dists, peaks_list, title):
    """Wyświetl obraz krawędzi z nałożonymi liniami dla różnych liczb maksimów.
    ↪ """
    n = len(peaks_list)
    fig, axes = plt.subplots(1, n, figsize=(5 * n, 5))
    if n == 1:
        axes = [axes]
    for ax, n_peaks in zip(axes, peaks_list):
        _, a_peaks, d_peaks = hough_line_peaks(H, angles, dists,
        ↪ num_peaks=n_peaks)
        ax.imshow(edge_img, cmap='gray')
        draw_hough_lines(ax, a_peaks, d_peaks, edge_img.shape, color='cyan')
        ax.set_title(f'{title}: {n_peaks} maksimów')
        ax.axis('off')
    plt.tight_layout()
    plt.show()

for label, fname in names:
    img = load_gray(image_path(fname))
    edges = edges_canny(img)
    H, angles, dists = hough_transform(edges)
    compare_peaks(edges, H, angles, dists, [5, 10, 20], label.capitalize())
```

Ez: 5 maksimów



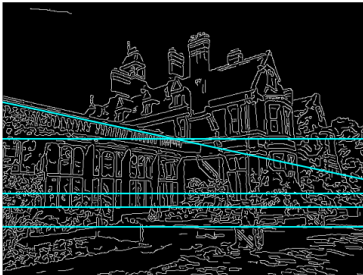
Ez: 10 maksimów



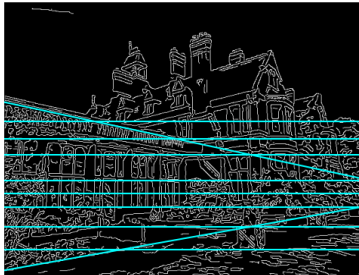
Ez: 20 maksimów



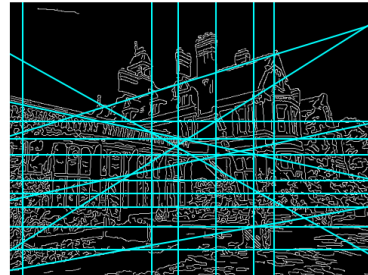
Dom: 5 maksimów



Dom: 10 maksimów



Dom: 20 maksimów



2.1.4 Wnioski

Z małą liczbą maksimów otrzymujemy tylko najbardziej dominujące kontury, natomiast większa liczba pozwala odwzorować więcej szczegółów, ale pojawiają się też linie nieistotne lub powtarzające się.

2.2 Zadanie 2.

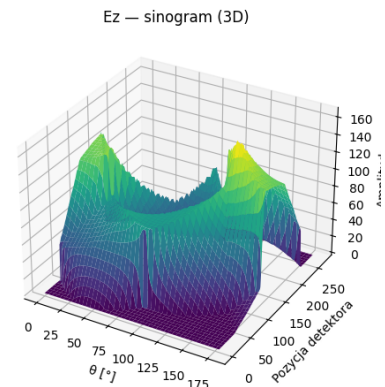
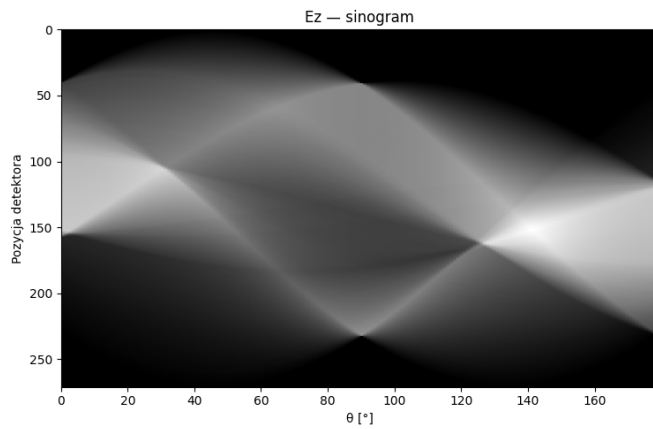
Oblicz transformatę Radona obrazów `ez.bmp`, `dom.bmp` oraz ich wersji konturowych. Przyjmij kwant kąta $\theta = 1^\circ$.

- Wyświetl transformaty jako wykres 3D oraz obrazy w poziomach szarości. Porównaj transformaty Radona obrazów konturowych z transformatami Hougha.
- Odtwórz obrazy za pomocą odwrotnej transformaty Radona (`iradon`) dla kwantów kąta 1° , 5° i 10° . Wyświetl i porównaj wyniki.

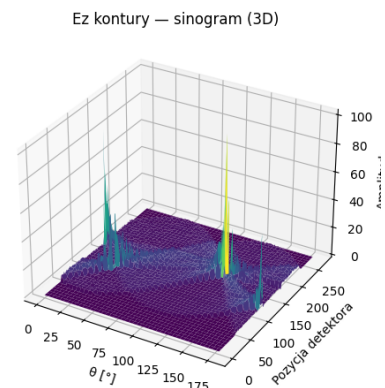
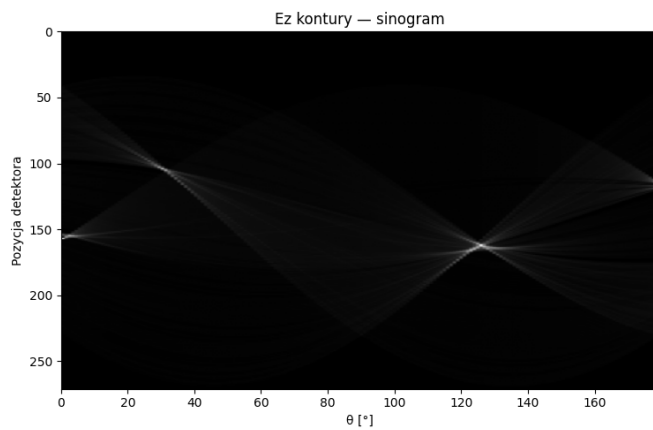
2.2.1 a) Transformata Radona

```
[4]: def crop_to_shape(img, shape):  
    """Przytnij obraz do podanego kształtu, wycinając środkowy fragment."""  
    h, w = shape  
    y0 = (img.shape[0] - h) // 2  
    x0 = (img.shape[1] - w) // 2  
    return img[y0:y0 + h, x0:x0 + w]  
  
def radon_transform(img, step=1):  
    """Oblicz transformatę Radona z zadanyim krokiem kąta."""  
    theta = np.arange(0, 180, step)  
    sinogram = radon(img, theta=theta, circle=False)  
    return sinogram, theta  
  
def plot_radon(sinogram, theta, title):  
    """Wyświetl sinogram jako obraz i wykres 3D."""  
    rho = np.arange(sinogram.shape[0])  
    T, R = np.meshgrid(theta, rho)  
    fig = plt.figure(figsize=(14, 5))  
    ax1 = fig.add_subplot(1, 2, 1)  
    ax1.imshow(sinogram, aspect='auto', cmap='gray',  
               extent=[theta[0], theta[-1], rho[-1], rho[0]])  
    ax1.set_title(f'{title} - sinogram')  
    ax1.set_xlabel('\u03b8 [°]')  
    ax1.set_ylabel('Pozycja detektora')  
    ax2 = fig.add_subplot(1, 2, 2, projection='3d')  
    ax2.plot_surface(T, R, sinogram, cmap='viridis', edgecolor='none')  
    ax2.set_title(f'{title} - sinogram (3D)')  
    ax2.set_xlabel('\u03b8 [°]')  
    ax2.set_ylabel('Pozycja detektora')  
    ax2.set_zlabel('Amplituda')  
    plt.tight_layout()  
    plt.show()  
  
variants = [('ez', 'ez.bmp'), ('dom', 'dom.bmp')]  
for label, fname in variants:  
    img = load_gray(image_path(fname))  
    edges = edges_canny(img)  
    for name, im in [(label, img), (f'{label} kontury', edges.astype(float))]:  
        sinogram, theta = radon_transform(im, step=1)  
        print(f'{name}: sinogram shape = {sinogram.shape}')  
        plot_radon(sinogram, theta, name.capitalize())
```

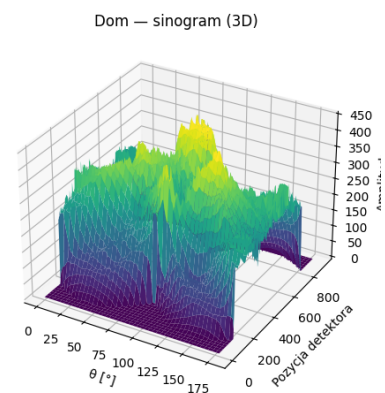
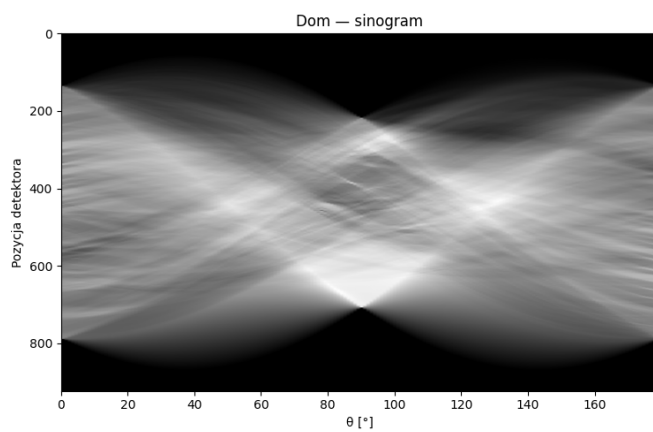
ez: sinogram shape = (272, 180)



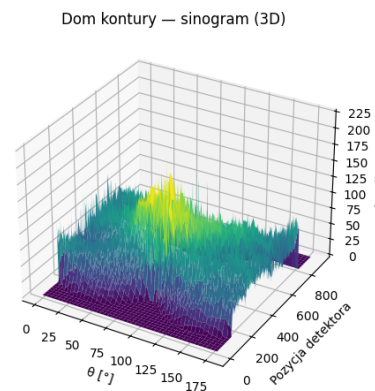
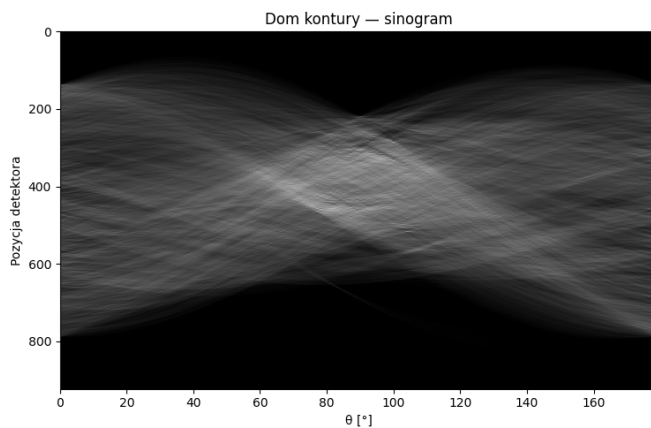
ez kontury: sinogram shape = (272, 180)



dom: sinogram shape = (924, 180)



dom kontury: sinogram shape = (924, 180)



2.2.2 Wnioski

Sinogramy obrazów konturowych są podobne do transformat Hougha, ponieważ obie metody akumulują wkład wzdłuż prostych. Radon uwzględnia jednak amplitudy pikseli, a Hough działa na binarnych krawędziach.

2.2.3 b) Odtworzenie obrazów metodą iradon

```
[5]: def reconstruct(img, step):  
    """Oblicz transformatę Radona i odtwórz obraz za pomocą iradon."""  
    theta = np.arange(0, 180, step)  
    sinogram = radon(img, theta=theta, circle=False)  
    rec = iradon(sinogram, theta=theta, output_size=max(img.shape))  
    return crop_to_shape(rec, img.shape)  
  
def compare_reconstructions(img, edges, label):  
    """Porównaj oryginał, krawędzie i rekonstrukcje dla różnych kroków."""  
    steps = [1, 5, 10]  
    fig, axes = plt.subplots(2, len(steps) + 1, figsize=(4 * (len(steps) + 1), 8))  
    axes[0, 0].imshow(img, cmap='gray')  
    axes[0, 0].set_title(f'{label} - oryginał')  
    axes[0, 0].axis('off')  
    axes[1, 0].imshow(edges, cmap='gray')  
    axes[1, 0].set_title(f'{label} - krawędzie')  
    axes[1, 0].axis('off')  
    for i, step in enumerate(steps, start=1):  
        rec_orig = reconstruct(img, step)
```

```

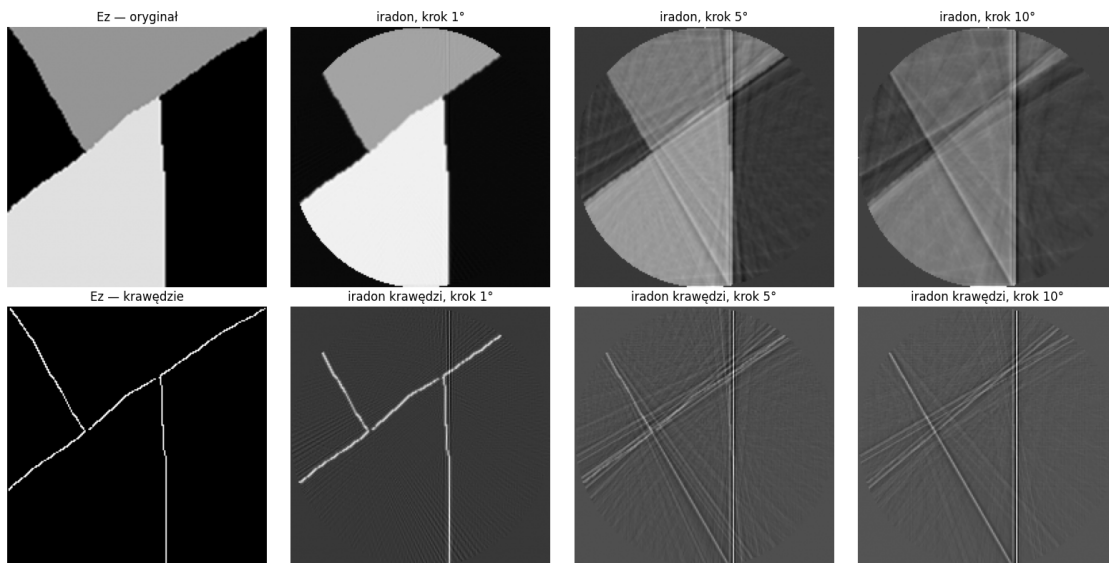
rec_edge = reconstruct(edges.astype(float), step)
axes[0, i].imshow(rec_orig, cmap='gray')
axes[0, i].set_title(f'iradon, krok {step}°')
axes[0, i].axis('off')
axes[1, i].imshow(rec_edge, cmap='gray')
axes[1, i].set_title(f'iradon krawędzi, krok {step}°')
axes[1, i].axis('off')
plt.tight_layout()
plt.show()

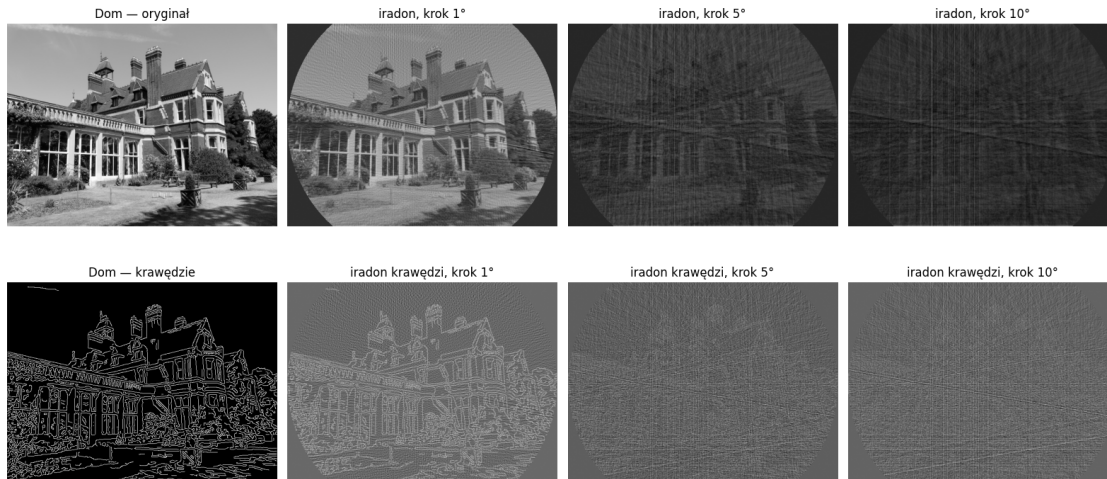
```

```

for label, fname in variants:
    img = load_gray(image_path(fname))
    edges = edges_canny(img)
    compare_reconstructions(img, edges, label.capitalize())

```





2.2.4 Wnioski

Rekonstrukcja z krokiem 1° zachowuje najwięcej szczegółów. Przy kroku 5° i 10° pojawiają się rozmycia i artefakty, ponieważ mniejsza liczba projekcji dostarcza mniej informacji do odwrotnej transformaty.

2.3 Zadanie 3.

Zrealizuj prostą aplikację do lokalizacji tęczówki (i znajdowania jej obrysu) w cyfrowym obrazie oka ludzkiego. Przykładowe obrazy to e7.jpg – e13.jpg.

```
[6]: def localize_iris(path):
    """Wykryj źrenicę i tęczówkę na obrazie oka za pomocą transformacji Hougha
    ↪ dla okręgów."""
    img = np.array(Image.open(path).convert('RGB'))
    gray = np.array(Image.open(path).convert('L'))
    blurred = gaussian(gray / 255.0, sigma=2)
    edges = canny(blurred, sigma=1.5, low_threshold=0.05, high_threshold=0.15)

    min_dim = min(gray.shape)
    radii = np.arange(max(5, min_dim // 10), min_dim // 2, max(1, min_dim //
    ↪ 50))
    h = hough_circle(edges, radii)
    _, cx, cy, rad = hough_circle_peaks(h, radii, min_xdistance=5,
    ↪ min_ydistance=5, num_peaks=2)
    circles = sorted(zip(rad, cx, cy), key=lambda c: c[0])

    result = img.copy()
    for i, (r, x, y) in enumerate(circles):
        color = (255, 0, 0) if i == 0 else (0, 255, 0)
        rr, cc = circle_perimeter(y, x, r, shape=result.shape)
```

```

        result[rr, cc] = color
    return result, edges, circles

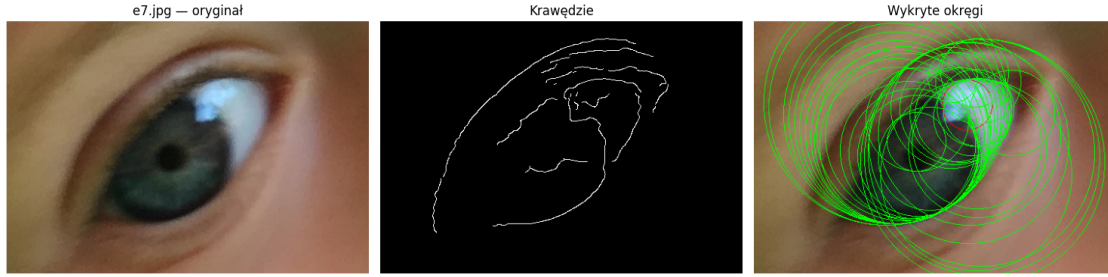
eye_files = [f'e{i}.jpg' for i in range(7, 14)]
for fname in eye_files:
    try:
        result, edges, circles = localize_iris(image_path(fname))
        print(f'{fname}: wykryte okręgi (r, x, y) = {circles}')
        fig, axes = plt.subplots(1, 3, figsize=(15, 5))
        axes[0].imshow(np.array(Image.open(image_path(fname)).convert('RGB')))
        axes[0].set_title(f'{fname} - oryginał')
        axes[0].axis('off')
        axes[1].imshow(edges, cmap='gray')
        axes[1].set_title('Krawędzie')
        axes[1].axis('off')
        axes[2].imshow(result)
        axes[2].set_title('Wykryte okręgi')
        axes[2].axis('off')
        plt.tight_layout()
        plt.show()
    except Exception as e:
        print(f'{fname}: błąd przetwarzania - {e}')

```

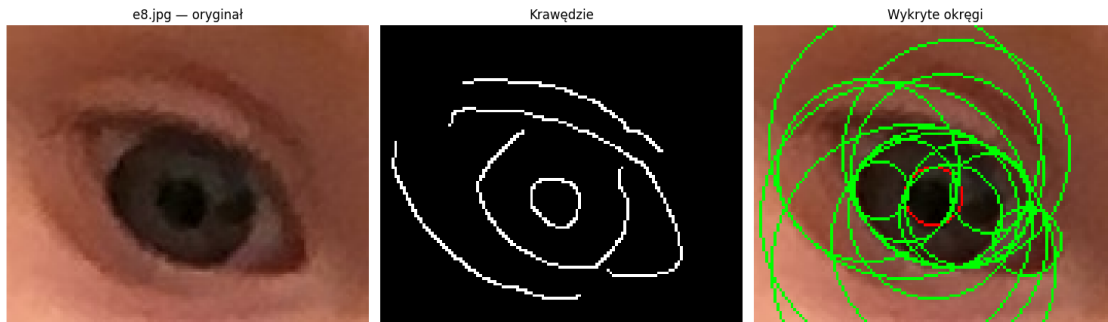
```

e7.jpg: wykryte okręgi (r, x, y) = [(np.int64(34), np.int64(290),
np.int64(113)), (np.int64(34), np.int64(270), np.int64(76)), (np.int64(40),
np.int64(221), np.int64(163)), (np.int64(46), np.int64(304), np.int64(123)),
(np.int64(46), np.int64(300), np.int64(70)), (np.int64(52), np.int64(296),
np.int64(130)), (np.int64(52), np.int64(301), np.int64(117)), (np.int64(58),
np.int64(305), np.int64(81)), (np.int64(58), np.int64(351), np.int64(124)),
(np.int64(64), np.int64(276), np.int64(126)), (np.int64(64), np.int64(283),
np.int64(127)), (np.int64(70), np.int64(239), np.int64(186)), (np.int64(70),
np.int64(222), np.int64(133)), (np.int64(76), np.int64(274), np.int64(118)),
(np.int64(82), np.int64(223), np.int64(182)), (np.int64(82), np.int64(268),
np.int64(117)), (np.int64(88), np.int64(303), np.int64(166)), (np.int64(88),
np.int64(217), np.int64(179)), (np.int64(94), np.int64(211), np.int64(175)),
(np.int64(94), np.int64(304), np.int64(172)), (np.int64(106), np.int64(195),
np.int64(168)), (np.int64(112), np.int64(194), np.int64(162)), (np.int64(112),
np.int64(209), np.int64(155)), (np.int64(118), np.int64(195), np.int64(156)),
(np.int64(118), np.int64(203), np.int64(152)), (np.int64(124), np.int64(194),
np.int64(150)), (np.int64(124), np.int64(293), np.int64(152)), (np.int64(130),
np.int64(192), np.int64(144)), (np.int64(130), np.int64(302), np.int64(208)),
(np.int64(136), np.int64(191), np.int64(138)), (np.int64(136), np.int64(292),
np.int64(165)), (np.int64(142), np.int64(189), np.int64(132)), (np.int64(148),
np.int64(188), np.int64(126)), (np.int64(154), np.int64(309), np.int64(178)),
(np.int64(160), np.int64(180), np.int64(116)), (np.int64(160), np.int64(310),
np.int64(184)), (np.int64(166), np.int64(179), np.int64(110)), (np.int64(166),
np.int64(311), np.int64(190))]

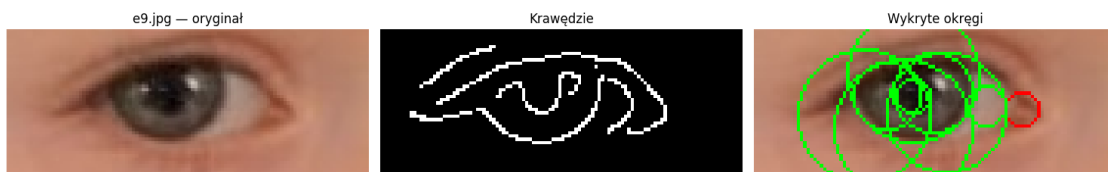
```



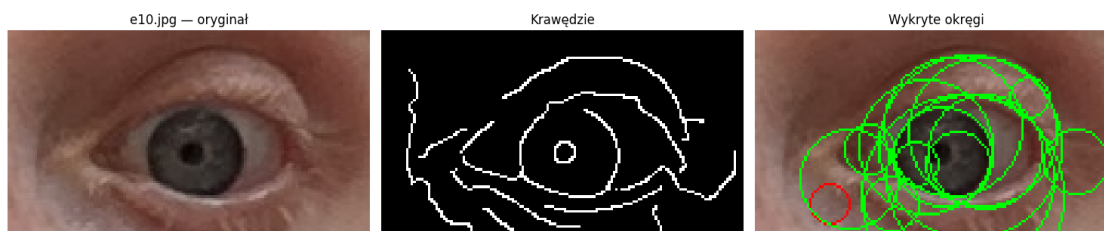
e8.jpg: wykryte okręgi (r, x, y) = [(np.int64(10), np.int64(63), np.int64(60)), (np.int64(10), np.int64(44), np.int64(58)), (np.int64(12), np.int64(96), np.int64(76)), (np.int64(14), np.int64(83), np.int64(59)), (np.int64(18), np.int64(69), np.int64(67)), (np.int64(18), np.int64(53), np.int64(58)), (np.int64(22), np.int64(75), np.int64(63)), (np.int64(30), np.int64(63), np.int64(66)), (np.int64(32), np.int64(40), np.int64(51)), (np.int64(34), np.int64(68), np.int64(51)), (np.int64(38), np.int64(65), np.int64(76)), (np.int64(40), np.int64(71), np.int64(45)), (np.int64(48), np.int64(53), np.int64(41)), (np.int64(48), np.int64(50), np.int64(68)), (np.int64(50), np.int64(56), np.int64(85))]



e9.jpg: wykryte okręgi (r, x, y) = [(np.int64(5), np.int64(81), np.int64(24)), (np.int64(5), np.int64(48), np.int64(20)), (np.int64(6), np.int64(70), np.int64(23)), (np.int64(10), np.int64(40), np.int64(20)), (np.int64(12), np.int64(54), np.int64(19)), (np.int64(18), np.int64(58), np.int64(25)), (np.int64(20), np.int64(48), np.int64(14)), (np.int64(20), np.int64(33), np.int64(26)), (np.int64(21), np.int64(45), np.int64(30))]



e10.jpg: wykryte okręgi (r, x, y) = [(np.int64(8), np.int64(30), np.int64(70)), (np.int64(8), np.int64(111), np.int64(27)), (np.int64(9), np.int64(65), np.int64(54)), (np.int64(10), np.int64(46), np.int64(48)), (np.int64(11), np.int64(55), np.int64(70)), (np.int64(13), np.int64(130), np.int64(53)), (np.int64(13), np.int64(82), np.int64(54)), (np.int64(20), np.int64(76), np.int64(51)), (np.int64(20), np.int64(38), np.int64(60)), (np.int64(23), np.int64(93), np.int64(49)), (np.int64(27), np.int64(69), np.int64(47)), (np.int64(28), np.int64(80), np.int64(39)), (np.int64(30), np.int64(92), np.int64(41)), (np.int64(34), np.int64(87), np.int64(60)), (np.int64(37), np.int64(87), np.int64(47)), (np.int64(41), np.int64(84), np.int64(67))]

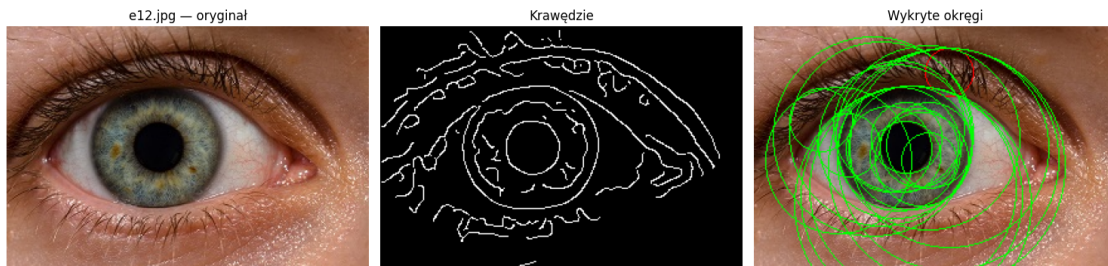


e11.jpg: wykryte okręgi (r, x, y) = [(np.int64(7), np.int64(52), np.int64(28)), (np.int64(7), np.int64(47), np.int64(64)), (np.int64(9), np.int64(32), np.int64(24)), (np.int64(9), np.int64(40), np.int64(26)), (np.int64(10), np.int64(32), np.int64(39)), (np.int64(12), np.int64(46), np.int64(49)), (np.int64(13), np.int64(69), np.int64(42)), (np.int64(15), np.int64(40), np.int64(51)), (np.int64(16), np.int64(83), np.int64(37)), (np.int64(17), np.int64(45), np.int64(20)), (np.int64(21), np.int64(77), np.int64(40)), (np.int64(23), np.int64(77), np.int64(33)), (np.int64(24), np.int64(56), np.int64(41)), (np.int64(27), np.int64(49), np.int64(42)), (np.int64(32), np.int64(77), np.int64(24)), (np.int64(32), np.int64(53), np.int64(34)), (np.int64(34), np.int64(65), np.int64(35)), (np.int64(37), np.int64(59), np.int64(35))]



e12.jpg: wykryte okręgi (r, x, y) = [(np.int64(19), np.int64(155), np.int64(36)), (np.int64(22), np.int64(139), np.int64(106)), (np.int64(25), np.int64(53), np.int64(74)), (np.int64(25), np.int64(75), np.int64(94)), (np.int64(28), np.int64(121), np.int64(88)), (np.int64(28), np.int64(120),

```
np.int64(103)), (np.int64(31), np.int64(131), np.int64(96)), (np.int64(34),
np.int64(107), np.int64(97)), (np.int64(34), np.int64(139), np.int64(94)),
(np.int64(37), np.int64(133), np.int64(105)), (np.int64(49), np.int64(124),
np.int64(96)), (np.int64(49), np.int64(115), np.int64(97)), (np.int64(52),
np.int64(115), np.int64(79)), (np.int64(58), np.int64(67), np.int64(106)),
(np.int64(61), np.int64(112), np.int64(91)), (np.int64(73), np.int64(101),
np.int64(85)), (np.int64(73), np.int64(138), np.int64(88)), (np.int64(79),
np.int64(94), np.int64(87)), (np.int64(79), np.int64(155), np.int64(96)),
(np.int64(82), np.int64(126), np.int64(129)), (np.int64(88), np.int64(125),
np.int64(135)), (np.int64(88), np.int64(154), np.int64(105)), (np.int64(94),
np.int64(126), np.int64(141)), (np.int64(94), np.int64(153), np.int64(111))]
```



e13.jpg: wykryte okręgi (r, x, y) = [(np.int64(5), np.int64(36), np.int64(34)),
(np.int64(5), np.int64(45), np.int64(30)), (np.int64(6), np.int64(74),
np.int64(24)), (np.int64(10), np.int64(30), np.int64(29)), (np.int64(13),
np.int64(68), np.int64(30)), (np.int64(13), np.int64(60), np.int64(30)),
(np.int64(16), np.int64(32), np.int64(23)), (np.int64(19), np.int64(59),
np.int64(36)), (np.int64(19), np.int64(65), np.int64(24)), (np.int64(26),
np.int64(64), np.int64(17)), (np.int64(26), np.int64(65), np.int64(43))]



2.3.1 Wnioski

Transformacja Hougha dla okręgów pozwala wykryć źrenicę i tęczówkę mimo zmiennego oświetlenia. Jakość detekcji zależy od zakresu promieni oraz parametrów detekcji krawędzi; dla bardzo małych obrazów zakres należy odpowiednio zawęzić.